IOT LABORATORY · END-SEMESTER VIVA

SignTalk Viva Survival Guide

Everything you need to explain our Indian Sign Language phrase translator, even if you did not write a single line of it.

Nishu Shukla
23BCP262

Ayaan Goel
23BCP264

Harsh Bhavsar
23BCP251

Submitted to **Dr. Ketan Sable** · PDEU, Gandhinagar

Contents

0 Read this first

1 The big picture (everyone learns this)

2 IoT basics and the hardware

3 Networking, HTTPS and the web server

4 MQTT, the heart of the IoT part

5 ESP32, wiring and electronics

6 Computer vision: MediaPipe and landmarks

7 The AI model, training and results

8 Data collection and Teach mode

9 The phone web app and hands-free mode

10 Results, limitations and honesty questions

11 "Why did you not use...?" The alternatives round

12 Demo script and when things go wrong

13 Know our files (what each one does)

14 Quick-fire round and trap questions

15 Cute glossary (one line each)

145 questions with ready answers, plus the 10-step story, numbers cheat sheet, demo script and file map.

0 Read this first

HOW TO USE THIS GUIDE Read sections 1 to 3 together once (30 minutes). Then each person reads their own topics in depth and everybody skims the rest. Every question has a short **"say"** answer that you can speak almost word for word, and a **"remember"** line with the one idea to keep in your head. The stars show how hard the question is: ★ easy, ★★ medium, ★★★ tricky.

Golden rule 1: be honest. If you do not know, say "we did not test that, but we think it would work because...". Professors forgive honesty and punish bluffing.

Golden rule 2: keep it simple. Use the everyday-words version first. Only add the technical term if they ask for more.

Golden rule 3: know our limits. One signer, 12 phrases, private Wi-Fi, no deaf-user testing. Saying the limits yourself shows maturity (Topic 10).

Golden rule 4: always bring it back to IoT. Sensing, connectivity, edge processing, messaging, actuation. If you are stuck, talk about those five words.

SUGGESTED SPLIT OF WHO EXPLAINS WHAT **Person A: the AI part** (Topics 6, 7, 8): landmarks, model, data, accuracy. **Person B: the network part** (Topics 3, 4, 9): Pi web server, HTTPS, MQTT. **Person C: the hardware part** (Topics 2, 5, 12): ESP32, wiring, demo. Everybody must be able to give the 2-minute pitch and answer Topic 1 and Topic 11 (alternatives). Decide the names among yourselves.

1 The big picture (everyone learns this)

IN SIMPLE WORDS A deaf person signs a phrase in front of a phone camera. The phone finds the hands and body as dots (landmarks) and sends only those numbers to a Raspberry Pi over Wi-Fi. The Pi runs our trained neural network, which says which phrase it was. The phone then speaks the phrase. At the same time the Pi publishes the result through an MQTT broker to an ESP32 board that shows the phrase on a small screen, lights an LED, beeps, and has an emergency button.

30-SECOND PITCH (MEMORISE THIS) "Deaf people who use Indian Sign Language cannot easily talk to people who do not know it. Our project, SignTalk, recognises 12 everyday ISL phrases and speaks them aloud. A phone browser is the camera and speaker, a Raspberry Pi 5 is the edge server that runs our trained model, and an ESP32 node shows the result on an OLED with LEDs and a buzzer, connected through MQTT. Everything works on a local Wi-Fi network with no cloud."

2-MINUTE PITCH (ADD THESE POINTS) 1) **Problem:** communication barrier, and almost all public models are for ASL not ISL. 2) **Our data:** we recorded 603 clips of 12 ISL phrases and trained our own model. 3) **Why landmarks:** the AI sees dots, not faces or backgrounds, so it is small, fast and private. 4) **Architecture:** phone = sensing and speaker, Pi = edge brain and MQTT broker, ESP32 = display and alert. 5) **Results:** 99.2 % in cross-validation, 2 to 3 ms per prediction on the Pi. 6) **Honest limits:** one signer, not tested with deaf users, MQTT without TLS. 7) **Future:** more signers, more phrases, TLS, more ESP32 nodes."

SignTalk IoT architecture: sensing, network, edge and actuation

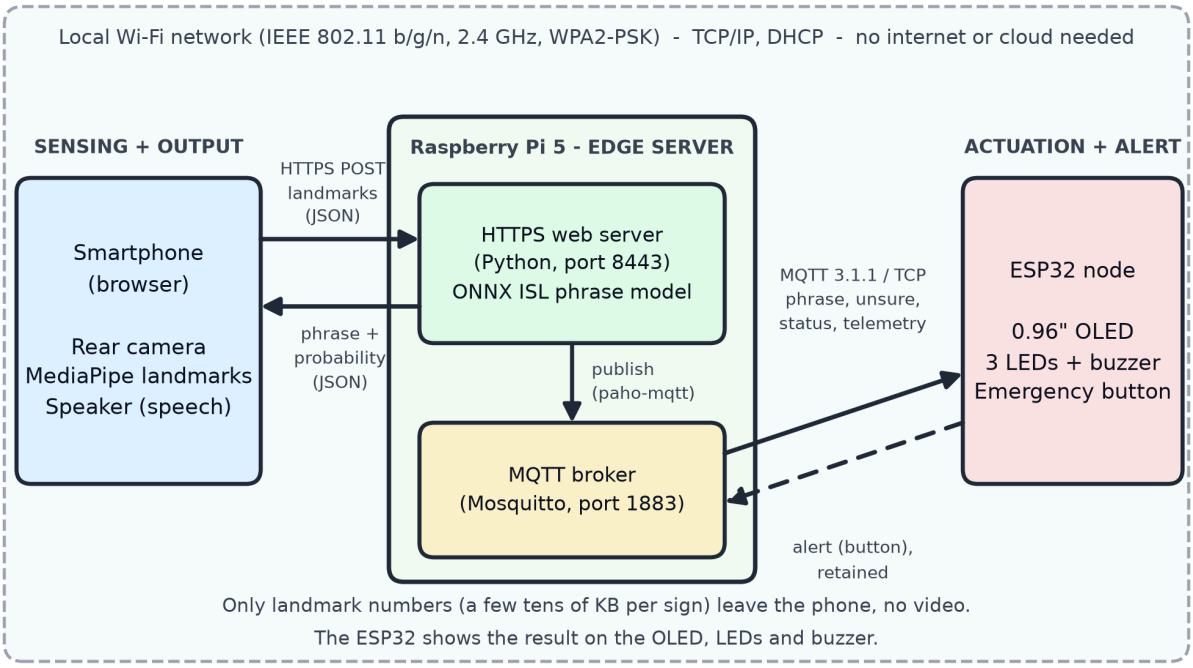


Figure 1: The whole system on one page. Learn to draw this on the board.

WHAT HAPPENS WHEN SOMEONE SIGNS "THANK YOU" (THE 10 STEPS)

1. The phone opens the web page from the Pi (HTTPS, port 8443) and starts the rear camera.
2. MediaPipe in the browser finds hand and body landmarks in every frame (about 29 fps).
3. The page notices a raised hand for 0.25 s: "a sign has started" (hands-free mode).
4. It collects the window from 1 s before to 2 s after, and sends only the landmark numbers (tens of KB, no video) to the Pi (`/api/predict`).
5. The Pi turns them into 32 frames of 198 numbers and runs the ONNX model (2 to 3 ms).
6. The answer comes back: best phrase and its probability. If it is above 55 % it is accepted.
7. The phone speaks the phrase with its text-to-speech voice and shows it big on screen.
8. The web server also publishes the phrase to the MQTT broker (topic `signtalk/phrase` , QoS 1).
9. The ESP32 is subscribed: it shows the phrase and a confidence bar on the OLED, lights the green LED for 2 s and beeps once.
10. If someone presses the button on the ESP32: red LED, alarm and "!! HELP !!" for 60 s or until pressed again.

NUMBERS CHEAT SHEET (THEY LOVE NUMBERS)

Phrases	**12**	Clips in data set	**603** (50 each, Sorry has 53)
Accuracy (5-fold CV)	**99.2 %** (598/603)	DTW baseline	**98.5 %** (594/603)
Prediction time on Pi	**2 to 3 ms**	Model size (ONNX)	**538 KB**
Training time on Pi	**about 95 s** (final model)	Features per frame	**198** (98 pos + 98 vel + 2 flags)
Frames per sign	**32**	Landmarks from MediaPipe	**543** (468 face, 21+21 hands, 33 pose)
Ports	HTTPS **8443**, HTTP **8080**, MQTT **1883**	Confidence threshold	**55 %** (slider on phone)
ESP32 pins	LEDs **25, 26, 27**, buzzer **32**, button **33**, OLED **21 (SDA), 22 (SCL)**	LED resistor	**330 ohm** (about 4 mA)
OLED	**0.96 inch**, SSD1306, 128x64, I2C **0x3C**	Telemetry	every **5 s**; alert auto-clears after **60 s**

2 IoT basics and the hardware

IN SIMPLE WORDS IoT means everyday objects that sense something, talk over a network, and do something useful, usually without a person in the loop. Our project has a sensor (camera, button), a network (Wi-Fi), a brain at the edge (Raspberry Pi) and actuators (speaker, OLED, LEDs, buzzer).

Edge computing = processing near the data (on the Pi) instead of in a far-away cloud

Sensor = measures the world (camera, button) **Actuator** = does something (LED, buzzer, speaker, display)

Node = one small device in the network (the ESP32)

Q1 Why is this project IoT and not just a machine-learning project? ★☆☆

SAY Because several connected devices cooperate over a network. The phone senses, the Pi processes at the edge, an MQTT broker moves messages, and the ESP32 acts by displaying, lighting LEDs and sounding a buzzer. The ML model is one part inside that system.

remember: sense, connect, process, act

Q2 What are the layers of our IoT system? ★☆☆

SAY Sensing layer: phone camera and the emergency button. Network layer: Wi-Fi with TCP/IP, HTTPS and MQTT. Edge layer: the Raspberry Pi with the model, web server and broker. Application and actuation layer: phone speaker and the ESP32 with OLED, LEDs and buzzer.

Q3 What is edge computing and why did you use it? ★★☆

SAY Edge computing means doing the processing close to where data is produced. Our Pi does the recognition on the local network, so there is no internet delay, no cloud cost, it works offline, and no video leaves the phone, which is good for privacy.

remember: low latency, offline, private

Q4 What does the Raspberry Pi do in your project? ★☆☆

SAY Four jobs: it serves the web page, it runs the trained ISL model (ONNX Runtime), it runs the Mosquitto MQTT broker, and it can retrain the model when we add new phrases.

Q5 What does the ESP32 do? Why not do that on the Pi? ★☆☆

SAY The ESP32 is a small Wi-Fi microcontroller. It subscribes to MQTT and drives the OLED, three LEDs, the buzzer and the emergency button. We first connected the LEDs to the Pi's GPIO, then moved them to the ESP32 so the output node is a separate wireless device that can be placed near the user, and the project shows real device-to-device IoT.

Q6 What is the difference between a Raspberry Pi and an ESP32? ★★☆

SAY The Pi is a small Linux computer (quad-core, gigabytes of RAM) that can run Python and an ML model. The ESP32 is a microcontroller (about 520 KB RAM) that runs one program, uses little power, and is great for sensors, displays and switches. Pi = brain, ESP32 = hands and eyes.

Q7 **Which components did you use?** ★☆☆

SAY Raspberry Pi 5, a smartphone, a USB webcam (Logitech Brio 100) for recording data, an ESP32 board, a 0.96-inch SSD1306 OLED, green, amber and red LEDs with 330 ohm resistors, a buzzer, a push button, a breadboard and jumper wires, and a Wi-Fi network.

Q8 **Why does the Raspberry Pi 5 need no speaker in your design?** ★★☆

SAY The Pi 5 has no 3.5 mm audio jack, and the phone already has a speaker and a good text-to-speech voice. So the phone speaks the phrase. A USB or Bluetooth speaker on the Pi would only be needed for our Option A design.

Q9 **Why did you take a phone as the camera instead of a camera module on the Pi?** ★★☆

SAY No extra hardware (everyone has a phone), the camera can be placed near the signer, several phones can use one Pi, only landmark numbers are sent (privacy and bandwidth), the phone speaks the answer, and the Pi load stays tiny. The trade-off is that a phone and a network link are needed. See Topic 11 for Option A.

remember: no extra hardware, portable, multi-user, private

3 Networking, HTTPS and the web server

IN SIMPLE WORDS Everything talks over one Wi-Fi network using TCP/IP. The phone talks to the Pi with HTTPS (a secure web connection). The Pi talks to the ESP32 with MQTT. The phone camera only works on a secure page, which is why the Pi uses HTTPS.

SSID = Wi-Fi network name | **DHCP** = router hands out IP addresses automatically

Port = numbered door on a computer (8443, 8080, 1883) | **REST + JSON** = ask a URL, get answer as text data

TLS = the encryption behind HTTPS

Q10 How are the devices connected? ★☆☆

SAY All three (phone, Pi, ESP32) join the same 2.4 GHz Wi-Fi network (IEEE 802.11, WPA2). The router gives them IP addresses by DHCP. The phone uses the Pi's IP address with HTTPS on port 8443, and the ESP32 uses it on port 1883 for MQTT.

Q11 Why HTTPS? Why not plain HTTP? ★★☆

SAY Browsers allow `getUserMedia`, the camera access, only on secure pages (HTTPS or localhost). So the Pi serves HTTPS with TLS. Plain HTTP on port 8080 is redirected to HTTPS.

remember: no HTTPS = no camera

Q12 What is a self-signed certificate? Why the browser warning? ★★☆

SAY A certificate proves the server's identity. Normally a trusted authority (like Let's Encrypt) signs it, but that needs a domain name and internet. Our Pi generates its own EC certificate with its IP addresses inside, so the browser shows a warning once. After the user accepts it, the connection is still encrypted.

Q13 Is a self-signed certificate secure? ★★★

SAY The traffic is encrypted, but the browser cannot confirm who the server is, so someone on the same network could in theory pretend to be the Pi. For a private lab network that is acceptable. For public use we would use a real certificate or install our own certificate authority on the phones.

Q14 What is REST and JSON? Which API calls did you make? ★★☆

SAY REST means using normal web requests (GET or POST to a URL) to do things. JSON is the text format for the data. Our endpoints: `/api/predict` (landmarks in, phrase out), `/api/teach` (save a new clip), `/api/retrain`, `/api/retrain_status`, `/api/config`, `/api/ping`, `/api/log`.

Q15 Why the Python standard library web server and not Flask or FastAPI? ★★☆

SAY To keep the Pi's load and dependencies small. The server needs only a few endpoints, so `ThreadingHTTPServer`, TLS and ONNX Runtime are enough. Flask or FastAPI are nicer for big apps and would also work.

Q16 How does the server stay fast on a Pi? ★★☆

SAY The model is tiny (538 KB), runs in 2 to 3 ms, files are gzip-compressed and cached with ETag, only landmark numbers are sent (not video), and the heavy part (MediaPipe) runs on the phone.

Q17 **What happens if the Wi-Fi drops?** ★★☆

SAY The phone page cannot reach the Pi and shows an error until Wi-Fi returns. The ESP32 reconnects by itself (auto-reconnect) and its red LED blinks slowly while it cannot reach the broker.

Q18 **Why is the ESP32 limited to 2.4 GHz?** ★★★

SAY The ESP32 Wi-Fi radio supports only 2.4 GHz (802.11 b/g/n). It cannot join a 5 GHz-only network. We faced this while commissioning, which is why the serial monitor prints the reason codes.

Q19 **Can someone outside the network use it?** ★★☆

SAY No, it is on a private LAN. For admin access the Pi also has SSH and Tailscale (a VPN), but the application does not use them. We deliberately keep port 1883 off the internet.

4 MQTT, the heart of the IoT part

IN SIMPLE WORDS MQTT is like a notice board with a post office. Programs do not talk directly; they send messages to a broker under a topic name. Anyone who has subscribed to that topic gets the message. The sender (publisher) does not need to know who is listening. Our broker is Mosquitto on the Pi.

Broker = the post office (Mosquitto)

Topic = the mailbox name, e.g. signtalk/phrase

Publish / subscribe = send / listen

QoS = delivery promise 0, 1 or 2

Retained = broker keeps the last message for new subscribers

Last will = message the broker sends if a client dies

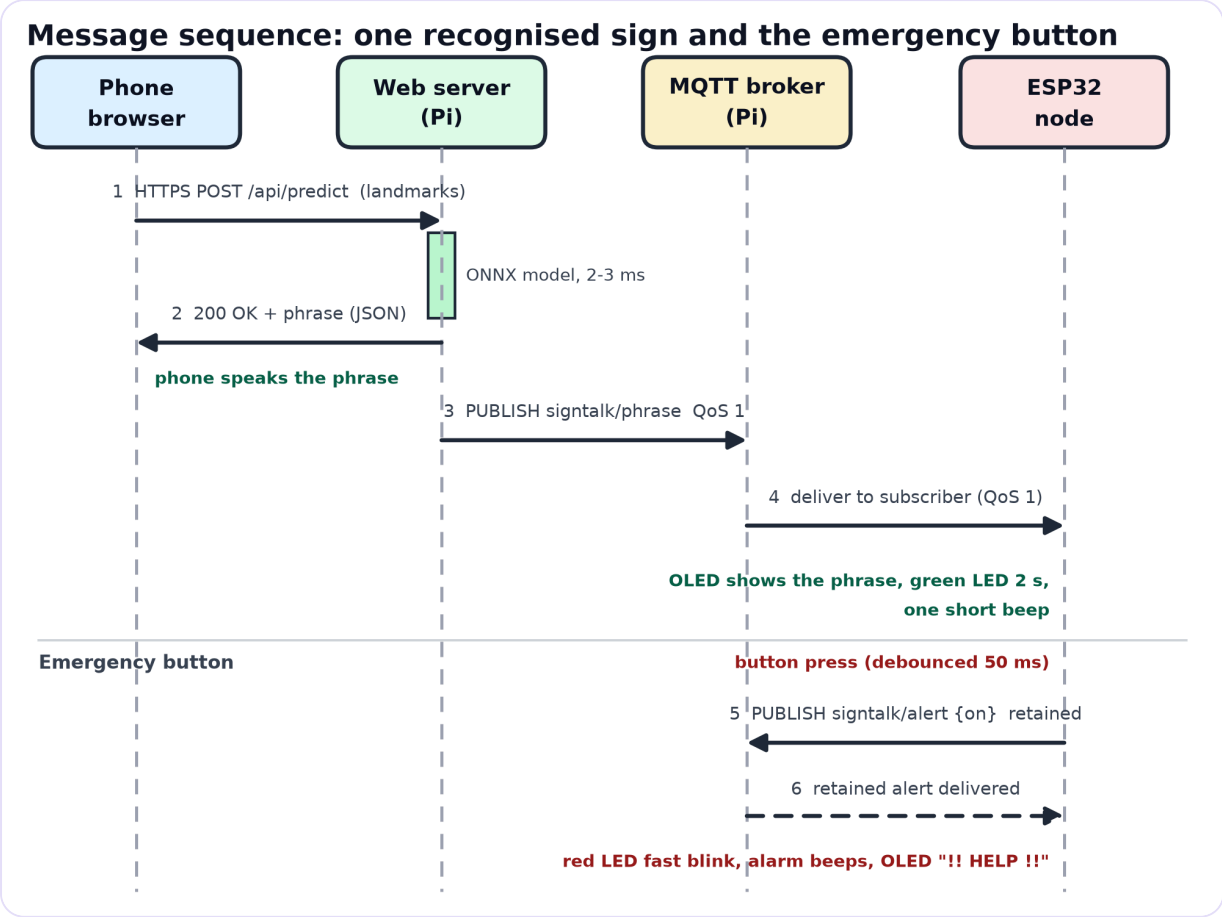


Figure 2: Message sequence. Learn steps 1 to 6 by heart.

OUR TOPICS (DRAW THIS TABLE IF ASKED)

Topic	From to	QoS	Retained	Meaning
`signtalk/phrase`	server to ESP32	1	no	an accepted phrase
`signtalk/unsure`	server to ESP32	0	no	sign seen but not accepted
`signtalk/status`	server to ESP32	1	yes	online / offline (last will)
`signtalk/telemetry`	server to ESP32	0	yes	Pi temp, load, counters every 5 s
`signtalk/alert`	ESP32 to all	0	yes	emergency button on/off

Q20 **What is MQTT in one sentence?** ★☆☆

SAY MQTT is a lightweight publish/subscribe messaging protocol over TCP, designed for small devices and unreliable networks, where a broker passes messages from publishers to subscribers by topic.

Q21 **What is a broker? Which one did you use and why?** ★☆☆

SAY A broker is the server that receives all messages and forwards them to the subscribers. We used Eclipse Mosquitto 2.0 on the Pi because it is free, tiny, very common and runs well on a Pi.

Q22 **Explain publish/subscribe. How is it different from normal HTTP?** ★★☆

SAY In HTTP the client asks and the server answers (request/response), and each client must know the server's address and ask again for new data. In pub/sub the sender and receiver never meet: the sender publishes to a topic and every subscriber gets it instantly. One event can reach many devices, and devices can come and go.

remember: decoupled: sender does not know receivers

Q23 **Explain QoS 0, 1 and 2. Which did you use?** ★★☆

SAY QoS 0 is at most once (fire and forget). QoS 1 is at least once (acknowledged, may duplicate). QoS 2 is exactly once (four-step handshake, slowest). We used QoS 1 for accepted phrases and status because they matter, and QoS 0 for unsure and telemetry because losing one is harmless. We did not need QoS 2.

Q24 **What are retained messages and why do you use them?** ★★☆

SAY A retained message is stored by the broker and given to every new subscriber immediately. We retain status, telemetry and alert, so an ESP32 that is switched on late instantly knows if the Pi is online, its temperature, and whether an alert is active.

Q25 **What is the Last Will and Testament?** ★★☆

SAY When the web server connects, it tells the broker "if I disappear without saying goodbye, publish `offline` on `signtalk/status`". So if the server crashes, the ESP32 learns it within the keep-alive time and blinks the red LED.

Q26 **What is keep-alive?** ★★☆

SAY A timer: the client must send some packet (or a ping) within the interval, 30 seconds on our server. If nothing arrives for about 1.5 times that, the broker declares the client dead and sends its last will.

Q27 **How did you secure the broker?** ★★☆

SAY Anonymous access is off, every client needs a user name and password, and the broker is meant for a private Wi-Fi. But the connection is not encrypted (no TLS), so the password travels in clear text on the LAN. For a real deployment we would use TLS on port 8883 and separate accounts with access rules (ACLs). This is also listed in our future scope.

remember: password yes, encryption no: be honest

Q28 **Anyone on the network could publish a fake alert. How would you stop that?** ★★★

SAY True, with one shared password and no TLS. We would add TLS, one account per device, and ACLs so that only the ESP32 account may publish to `signtalk/alert` and only the server account may publish the phrases.

Q29 **Why is the alert published with QoS 0 from the ESP32?** ★★☆

SAY The PubSubClient library on the ESP32 can only publish with QoS 0 (it can subscribe at QoS 0 or 1). We still set the retained flag. A lost alert on a very bad Wi-Fi link is a limitation we mention in the report.

Q30 **What is persistence in Mosquitto?** ★★☆

SAY It saves retained messages and subscriptions to disk, so they survive a broker restart. We turned it on.

Q31 **Which libraries did you use for MQTT?** ★★☆

SAY On the Pi, `paho-mqtt` (Python). On the ESP32, `PubSubClient`. The Pi side connects asynchronously with automatic reconnect, and publishing never raises an error, so a broker failure cannot stop the recognition.

Q32 **What happens if the broker is down?** ★★☆

SAY Recognition still works on the phone because the web server catches the failure and keeps running. The ESP32 cannot connect, so its OLED shows "Connecting to Pi" and the red LED blinks slowly. When the broker returns, everything reconnects by itself.

Q33 **How many messages can your broker handle? Is it scalable?** ★★★

SAY Our traffic is tiny: a few small messages per sign and one telemetry message every 5 s. Mosquitto can handle thousands of clients and messages per second on a Pi-class device, so the broker is not our bottleneck. We tested only one ESP32 node though.

5 ESP32, wiring and electronics

IN SIMPLE WORDS The ESP32 runs one Arduino program in a loop that never waits (no long delays). It connects to Wi-Fi and MQTT, listens for messages, reads the button, switches LEDs and the buzzer, and redraws the OLED. The wires are: OLED on two I2C wires, three LEDs through resistors, a buzzer, and a button to ground.

GPIO = general purpose pin you can switch on or off, or read

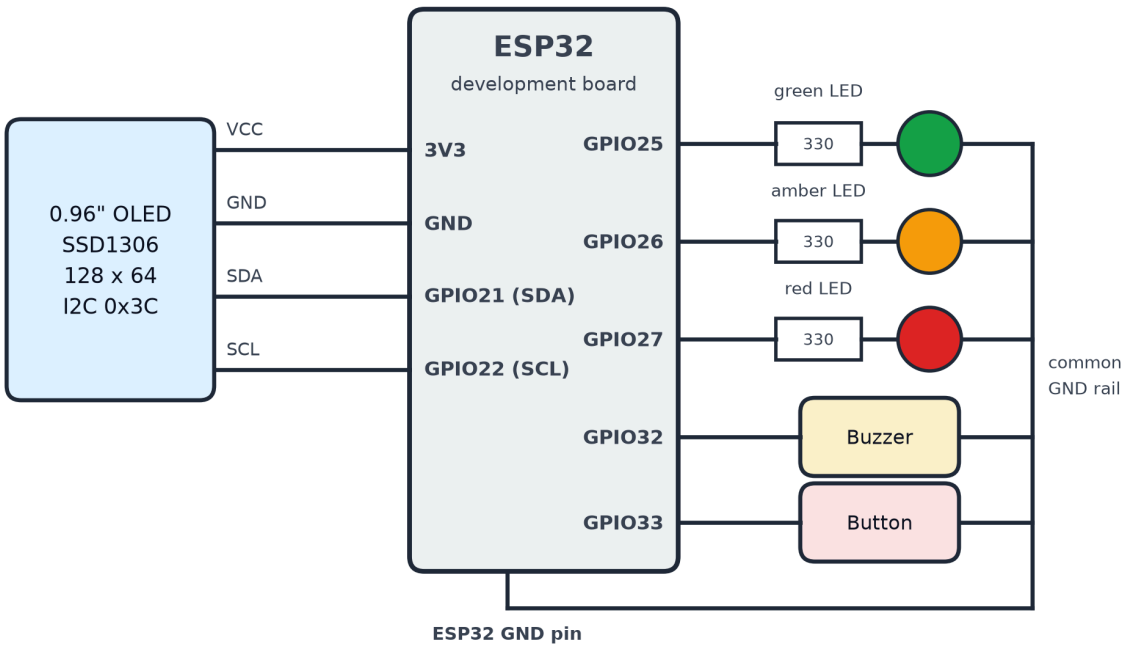
I2C = two-wire bus (SDA data, SCL clock) for small displays

Pull-up = resistor that keeps an input HIGH until the button pulls it LOW

Debounce = ignore the shaking of a button for a few ms

PWM = fast on/off to make a tone or dim an LED

ESP32 node wiring: OLED (I2C), LEDs, buzzer and emergency button



LED current = (3.3 V - about 2 V) / 330 ohm = about 4 mA.
The button uses the ESP32 internal pull-up (no resistor): pressed = LOW.

Figure 3: ESP32 wiring. You should be able to redraw this.

Q34 Tell me the connections of the ESP32. ★☆☆

SAY OLED: VCC to 3V3, GND to GND, SDA to GPIO21, SCL to GPIO22. Green LED on GPIO25, amber on GPIO26, red on GPIO27, each through a 330 ohm resistor to ground. Buzzer on GPIO32. Push button on GPIO33 with the other leg to ground.

remember: 25-26-27 LEDs, 32 buzzer, 33 button, 21/22 OLED

Q35 Why a resistor with each LED? How did you choose 330 ohm? ★★☆

SAY An LED has almost no resistance of its own, so without a resistor it draws too much current and burns out (and can damage the pin). The current is (3.3 V - about 2 V LED drop) / 330 ohm = about 4 mA, which is bright enough and safe for an ESP32 pin.

remember: I = (3.3 - 2) / 330 = about 4 mA

Q36 What is I2C? Why is the OLED on I2C? ★★☆

SAY I2C is a two-wire bus: SDA carries data and SCL is the clock. Many devices can share the same two wires, each with an address; our OLED is at 0x3C. It saves pins compared with SPI or a parallel display.

Q37 What is the SSD1306? ★★☆

SAY The driver chip inside the 0.96 inch OLED (128 x 64 pixels). We talk to it through the Adafruit SSD1306 and GFX libraries, which draw text and shapes into a buffer and then send it.

Q38 Why INPUT_PULLUP for the button? Why no external resistor? ★★☆

SAY If a button input is not connected to anything it floats and gives random readings. The ESP32 has an internal pull-up resistor that holds the pin HIGH. Pressing the button connects the pin to ground, so pressed means LOW.

remember: pressed = LOW

Q39 What is debouncing and how did you do it? ★★☆

SAY A mechanical button bounces for a few milliseconds and gives many false presses. In software we ignore changes for 50 ms after the last change and act only on the falling edge (HIGH to LOW).

Q40 What does "non-blocking" mean in your code? Why not use delay()? ★★★

SAY `delay()` freezes the whole program, so the button, MQTT and display would not work meanwhile. We use `millis()` and store time stamps (for example `greenUntil = now + 2000`), and the loop just compares the time. That lets LEDs, buzzer, Wi-Fi, MQTT, button and display all run together.

remember: millis() deadlines, no waiting

Q41 How does the ESP32 behave? Explain each LED. ★★☆

SAY Green LED 2 s and one beep when a phrase is recognised. Amber LED 1.5 s when a sign was seen but not accepted. Red LED blinks slowly when the Pi, broker or Wi-Fi is not reachable, and blinks fast with alarm beeps and a flashing "!! HELP !!" screen during an emergency alert.

Q42 How does the emergency button work end to end? ★★☆

SAY A debounced press toggles the alert on the ESP32 and publishes `{state: on}` to `signtalk/alert` as a retained message. Any subscriber, including the ESP32 itself, sees it. The alert switches itself off after 60 s or when the button is pressed again.

Q43 Active vs passive buzzer? ★★☆

SAY An active buzzer has its own oscillator, so DC makes it beep. A passive buzzer needs a tone (a square wave) from the program. Our firmware has a `PASSIVE_BUZZER` setting for that. A bare buzzer should be driven through a transistor so the pin is not overloaded.

Q44 What was the problem with the Wi-Fi connection on the ESP32 and how did you fix it? ★★☆

SAY At first the code called `WiFi.begin()` again every loop, which kept cancelling the connection attempt. We now start Wi-Fi once, use the ESP32's auto-reconnect, switch off Wi-Fi sleep, and print the disconnect reason on the serial monitor (201 = network not found, 204 or 15 = security handshake failed). That showed that the main router refused the node, so it joined another 2.4 GHz network.

Q45 **What happens at power-up?** ★★☆

SAY A self-test: each LED flashes once and the buzzer beeps once (a quick wiring check), then the OLED shows a start-up screen and Wi-Fi starts.

Q46 **Why do you use GPIO 25, 26, 27, 32, 33?** ★★★

SAY They are normal input/output pins. We avoided pins that matter during boot (strapping pins like 0, 2, 12, 15) and pins that can only be inputs (34 to 39).

Q47 **What libraries and tools did you use on the ESP32?** ★★☆

SAY Arduino IDE with the ESP32 board package (board ESP32 Dev Module), PubSubClient for MQTT, Adafruit SSD1306 and Adafruit GFX for the display. Serial monitor at 115200 baud for debugging.

6 Computer vision: MediaPipe and landmarks

IN SIMPLE WORDS Instead of giving the AI the full video, we first turn every frame into a handful of dots: points on the hands and body. A free Google tool called MediaPipe does that. The AI then only sees how the dots move. That makes it small, fast, private and independent of lighting, clothes and background.

Landmark = a tracked point such as a wrist or finger joint

MediaPipe = Google's tracking framework

Pose = body points (33)

Normalisation = making numbers comparable (shoulder width = 1)

Q48 What is MediaPipe? ★☆☆

SAY An open-source framework from Google for real-time perception, with ready models for hand tracking, body pose and face. It runs in the browser (JavaScript) and in Python. It gives us the landmark coordinates.

Q49 What are landmarks and how many do you get? ★★☆

SAY Points with x and y (and z) positions. MediaPipe's holistic layout has 543: 468 on the face, 21 for each hand and 33 on the body. For sign recognition we keep only 7 body points (nose, shoulders, elbows, wrists) and 21 points for each hand, so 49 points with x and y.

Q50 Why landmarks and not the raw video? ★★☆

SAY Smaller and faster to process, much less data to send, no dependence on lighting, background or clothes, works with small data sets, and privacy: the video never leaves the phone. Raw video models (3D CNNs) need far more data and a GPU.

remember: small, fast, private, robust

Q51 How do you make the data independent of distance from the camera? ★★★

SAY We shift the coordinates so the middle of the shoulders is the origin and divide by the shoulder width. So a person close to or far from the camera, or standing off-centre, gives similar numbers.

Q52 What is the input to the neural network exactly? ★★☆

SAY Each clip is resampled to 32 frames. Each frame has 198 numbers: 98 positions (49 points x 2), 98 velocities (difference from the previous frame, because motion matters in signing) and 2 flags saying whether the left and right hands were detected. Missing hands are marked as not detected.

Q53 Where does MediaPipe run: on the phone or on the Pi? ★★☆

SAY In the phone browser (MediaPipe Tasks Vision JS, self-hosted on the Pi so no internet is needed), at about 29 fps with the phone GPU. On the Pi it took 62 to 90 ms per frame (about 11 to 16 fps) and loads the CPU, which is why we moved it to the phone. For recording training clips MediaPipe ran on the Pi in Python (about 11 fps).

Q54 **Training used Python MediaPipe but the phone uses JavaScript MediaPipe. Is that a problem?** ★★★

SAY They are the same models and landmark layout, and we use the same version family, so features should match. We corrected for the phone's 16:9 image shape. We did see a mismatch in a headless browser test, but on a real phone the user tested it and it worked very well. A fully quantified phone accuracy test is future work.

remember: honest: works on phone, not yet measured numerically

Q55 **What if there are two people or no hands in the picture?** ★★☆

SAY The server rejects windows where no body or no hands are seen, and returns "no_body" or "no_hands". Two people in the frame can confuse the tracker, so we ask for one signer with head, shoulders and both hands in view.

Q56 **Why does the page draw dots on the video?** ★☆☆

SAY So the user can see what the system tracks and adjust their position (for example when hands are not detected).

7 The AI model, training and results

IN SIMPLE WORDS Our model reads 32 frames of numbers and says which of the 12 phrases it is, with a probability for each. Inside, small 1D-convolution layers pick up short movement patterns, a bidirectional GRU understands the order of those patterns over time, and a final layer gives 12 scores that are turned into probabilities. We trained it ourselves on 603 clips.

1D-CNN = finds short patterns in a sequence

GRU = a memory cell that reads a sequence

Softmax = turns scores into probabilities

Epoch = one pass through all training data

Overfitting = memorising training data, failing on new data

DTW = stretch-and-compare matching of sequences

Q57 What model did you train? ★☆☆

SAY A 1D-CNN plus bidirectional GRU network: two Conv1D layers (64 channels, kernel 5, batch norm, ReLU), one bidirectional GRU with 64 units, mean and max pooling over time, dropout 0.3 and a linear layer with 12 outputs. It is about 538 KB as an ONNX file.

Q58 Why a 1D-CNN with a GRU? ★★☆

SAY A sign is a sequence of hand and body positions over time. Convolutions detect short motion patterns, the GRU captures the longer order of events, and together they are accurate yet small enough for a Pi. They need much less data than a transformer.

Q59 What is a GRU? How is it different from an LSTM? ★★☆

SAY Both are recurrent networks that keep a memory while reading a sequence. A GRU has fewer gates (two) than an LSTM (three), so fewer parameters and faster. LSTM would probably work similarly, but we did not compare them.

Q60 What is bidirectional? ★★☆

SAY The GRU reads the sequence forwards and backwards and joins both. That is fine because we classify a complete recorded window, not a live stream word by word.

Q61 What is DTW and why did you also train it? ★★☆

SAY Dynamic Time Warping compares two sequences while stretching time, so a slow and a fast version of the same sign still match. We use the average of the three nearest clips as a score. It needs no training, so it is a strong, simple baseline. The neural network got 99.2 % against 98.5 % for DTW.

Q62 How did you train the network? ★★☆

SAY 80 epochs with the AdamW optimiser, a one-cycle learning-rate schedule and label smoothing 0.1. We augmented the data on the fly: small rotation (+/-8 degrees), scaling (0.9 to 1.1), shifting, noise, random time cropping and stretching, and brief loss of the hands. The final model trains in about 95 s on the Pi.

Q63 What is label smoothing? What is data augmentation? Why use them? ★★★

SAY Augmentation makes new training examples by slightly changing the existing ones, so the model sees more variety from 603 clips. Label smoothing stops the model being 100 % sure on training data (targets 0.9 instead of 1.0), which reduces overfitting and gives more honest probabilities.

Q64 **What is cross-validation? How did you evaluate?** ★★☆

SAY 5-fold stratified cross-validation: we split the clips in 5 parts keeping the phrase balance, train on 4 parts and test on the 5th, five times, so every clip is tested once while never being in its own training set. Result: 598 of 603 correct (99.2 %). The training accuracy is not an accuracy estimate.

Q65 **99 % looks too good. Is it real?** ★★★

SAY It is the honest cross-validation number on our data, but it is optimistic: all clips come from one signer and similar recording sessions. For a new person, day or camera we expect lower accuracy and we have not measured it. That is the main limitation of the project.

remember: say the limit before they do

Q66 **Which phrases were confused?** ★★☆

SAY Good morning was the weakest (46 of 50): twice mistaken for Good afternoon and twice for Sorry. Good afternoon was once mistaken for Sorry. The four phrases we added later (How are you, I am fine, Nice to meet you, Have a good day) had no errors.

Q67 **What is the confidence threshold?** ★★☆

SAY The network outputs a probability for each phrase. If the best one is above 55 %, we accept and speak it, otherwise we show "not sure". When the network was right the average confidence was 0.91 and when wrong 0.55, so the threshold removes many wrong answers. We first used 80 %, which was too strict on the phone, so we lowered it and added a slider.

Q68 **What is ONNX and why did you use it?** ★★☆

SAY ONNX is a standard file format for trained models. We train in PyTorch, export to ONNX, and run it with ONNX Runtime on the Pi, which is fast and avoids installing the large PyTorch library on the server. The exported model matches PyTorch to about 5x10^-7.

Q69 **Why only 12 phrases? Can it recognise sentences?** ★★☆

SAY It is an isolated-phrase classifier: one phrase per window. Continuous signing needs far more annotated data and a different model (sequence-to-sequence with CTC). New phrases can be added with Teach mode and retraining.

Q70 **How is a phrase not in the list handled?** ★★☆

SAY The model always picks the closest of the 12, but if its probability is below the threshold we show "not sure". There is no "no sign" class yet; adding one is in our future work.

Q71 **What did you try before this?** ★★★

SAY We first ran pretrained American Sign Language models (PopSign-250 and an ASL Citizen ST-GCN with 2,731 signs) on the Pi. They ran but recognise ASL, not ISL, and were not reliable on live camera input. So we changed to our own ISL model.

Q72 **Did you overfit? How do you know?** ★★★

SAY With 603 clips and a small network, overfitting is a real risk. We use dropout, augmentation, label smoothing and, most importantly, cross-validation, where the test clips are never in the training set. A gap between training accuracy and cross-validation accuracy would show overfitting.

8 Data collection and Teach mode

IN SIMPLE WORDS We recorded the signs ourselves with a webcam connected to the Pi, using a small recorder web page that counts down, records 3 seconds and saves the landmark numbers. Later we added a Teach mode on the phone page: record new clips, press Retrain, and the Pi trains a new model in the background and loads it automatically.

Q73 How did you collect the data? ★☆☆

SAY A recorder page on the Pi shows the USB webcam, gives a 3-second countdown, records a 3-second clip and saves the MediaPipe landmarks of every frame, plus a video and a metadata file. The Logitech Brio 100 runs at 1280x720 and is downscaled to 640x360; about 11 fps gives about 34 frames per clip.

Q74 How big is the data set? ★☆☆

SAY 12 phrases and 603 clips: 50 clips for each phrase (Sorry has 53). The first eight phrases were recorded first and extended to 50 clips; four more (How are you, I am fine, Nice to meet you, Have a good day) were added with 50 clips each.

Q75 Where did you get the signs from? ★★☆

SAY From the ISLRTC (Indian Sign Language Research and Training Centre) reference dictionary and videos. The signer is not an expert, so the signs have not been verified by an ISL expert or deaf users. We say this openly.

remember: limit: not expert-verified

Q76 Why only one signer? ★★☆

SAY Time and resources. It is the main weakness: accuracy for other people is not measured. Recording more signers is the first item in future scope.

Q77 What is Teach mode? ★★☆

SAY A tab on the phone page. The user types or selects a phrase, records clips with the phone camera, and presses Retrain. The Pi adds the clips to the data set and retrains in the background with low priority. The old model is backed up and the new one is loaded without restarting the server.

Q78 How can you add more phrases later? ★★☆

SAY Record about 50 clips of the phrase (with the recorder or Teach mode), run `train_phrases.py` (or press Retrain), and the new model appears. The new phrase needs a display and speak text in `phrases_display.json`.

Q79 How long does retraining take? ★★★

SAY About 1.5 minutes on the Pi for the final model without cross-validation, and about 11.5 minutes with the full 5-fold cross-validation.

9 The phone web app and hands-free mode

IN SIMPLE WORDS The page is plain HTML, CSS and JavaScript, no app install. It opens the rear camera, tracks landmarks, waits for you to raise a hand, sends the window to the Pi, and reads the answer aloud. It uses the browser's own camera API and text-to-speech.

Q80 Why a web app and not an Android/iOS app? ★☆☆

SAY No installation, works on any phone with a modern browser, one code base, and the Pi can update it instantly. A native app could use the camera and GPU more directly and work offline, but it is more work and needs app-store or install steps.

Q81 Explain hands-free mode. ★★☆

SAY The page watches the wrist height. When a wrist is above the lap (about 0.8 shoulder widths above the shoulder line) for 250 ms, it assumes a sign has started and sends the window from 1 s before to 2 s after to the Pi. It needs at least 12 frames with the hand up, the hands must go down for 0.6 s before the next sign, and there is a 1.2 s cooldown. A Tap to sign mode is also available.

Q82 How does the phone speak? ★★☆

SAY With the browser's Web Speech API (speech synthesis). The Pi sends back the phrase text and the phone's own voice reads it. Settings let the user pick the voice and volume.

Q83 How is the camera accessed? ★★☆

SAY With the browser's Media Capture API (`getUserMedia`), which needs HTTPS. The user must press Allow. We request HD video (1280x720).

Q84 What does the server send back? ★★☆

SAY The best three phrases with probabilities, whether the best was accepted, the model time in ms, and a reason if rejected (too short, no body, no hands, or unsure).

Q85 What is the lag from sign to speech? ★★☆

SAY The model takes 2 to 3 ms and the network a few ms, so the delay is dominated by the sign itself (the window after the sign starts is 2 s) and the speech start. We did not measure the full end-to-end delay with a stopwatch, which we say as a limitation.

Q86 How does the web server handle many phones? ★★★

SAY It is multi-threaded and each prediction takes only a few ms, so several phones can share the Pi. We tested with one phone at a time; a load test is future work.

10 Results, limitations and honesty questions

IN SIMPLE WORDS Be proud of what works and calm about what does not. Professors often ask the hard question on purpose to see if you understand your own limits. Every limit below has a ready answer and a plan to fix it.

What works: End-to-end recognition on a phone, 99.2 % cross-validation, 2 to 3 ms inference, hands-free mode, Teach mode and retrain, MQTT broker, ESP32 node with OLED, LEDs, buzzer and emergency button, and the ESP32 verified on real hardware (OLED shows Wi-Fi, MQTT, Pi temperature and the phrases).

What is limited: One signer, 12 phrases, signs not expert-verified, no deaf-user testing, no 'no sign' class, MQTT not encrypted, only one ESP32 node, no end-to-end latency measurement, hotspot and auto-start at boot not done.

Q87 What are the limitations of your project? ★★☆

SAY One signer and 12 isolated phrases, signs not verified by an ISL expert, no tests with deaf users, no 'no sign' class so random motion can be recognised, MQTT without TLS, one ESP32 node, and the Pi must share a 2.4 GHz network with the ESP32. Accuracy for new people is not measured.

Q88 What will you improve if you get more time? ★★☆

SAY More signers and sessions, expert verification of the signs, a 'no sign' class, more phrases and then sentences, MQTT over TLS with per-device accounts, more ESP32 nodes, the Pi as its own Wi-Fi hotspot starting at boot, and a dashboard for telemetry.

Q89 Did you test with a deaf person? ★★★

SAY No. The signs were learned from ISLRTC videos by a non-signer, so they must be verified before real use. The report says this clearly.

Q90 How did you do the work? Who wrote the code? ★★★

SAY Answer truthfully and in line with the institute's rules. If AI tools or online examples were used, say so, and then show that you understand the design: you chose the problem, recorded the data, selected the architecture and can explain every file (see Topic 13). Understanding your project is what is judged.

remember: honest + able to explain every part

Q91 What would happen with a very different background or lighting? ★★☆

SAY Because the AI sees landmarks, not pixels, background and clothes matter little. Bad lighting still hurts MediaPipe's tracking and then the dots are wrong, so we ask for a well-lit scene.

Q92 What about privacy? ★★☆

SAY Video never leaves the phone. Only landmark numbers go to the Pi, which is on your own network, and nothing goes to the cloud.

Q93 Can this work for a different language or a new person? ★★★

SAY The pipeline works for any signs: record clips, retrain. A new signer needs new clips (or the model must be tested and fine-tuned) because accuracy for unseen people is not guaranteed.

11 "Why did you not use...?" The alternatives round

IN SIMPLE WORDS Professors love this question. The pattern of a good answer is always three beats: (1) it is a valid option, (2) here is why it did not fit our constraints, (3) here is when we would choose it. Never say the alternative is bad. Our constraints were: low cost, a Raspberry Pi and an ESP32 only, local network, small data set, real-time.

Q94 Why not use a cloud server (AWS/Google) instead of the Pi? ★★☆

SAY Valid, and it scales better. But it needs internet, adds delay and cost, and sends data away. The local Pi gives low latency, offline use and privacy, and shows edge computing. For thousands of users we would consider the cloud.

Q95 Why not put the camera and speaker directly on the Pi (Option A)? ★★☆

SAY That is a valid stand-alone design and we compared both. It needs extra hardware (camera, speaker, power), supports one user, the Pi 5 has no audio jack, and the Pi would run MediaPipe at 11 to 16 fps. The phone gives a better camera, speaker, portability and several users for free.

Q96 Why not an ESP32-CAM for the camera? ★★☆

SAY Its camera is low resolution and the chip cannot run MediaPipe, so we would have to stream video to the Pi, which loads the Pi and network and lowers accuracy. A phone camera is much better.

Q97 Why not sensor gloves with flex sensors? ★★☆

SAY Gloves can be accurate for finger shapes, but the signer must wear special hardware, they are costly, and they miss body position, arm movement and facial expression. A camera is non-intrusive.

Q98 Why not a deep video model (3D CNN, I3D, video transformer) on raw video? ★★★

SAY They can be very accurate but need large data sets (ours is 603 clips), a GPU, and are sensitive to background and lighting. Landmarks give a small, fast model that runs in 2 to 3 ms on a Pi.

Q99 Why not a Transformer or LSTM instead of CNN + GRU? ★★★

SAY Transformers need more data to shine and are heavier. An LSTM would probably perform similarly to the GRU but has more parameters. We chose a light model suited to a Pi and a small data set. We did not benchmark them against each other.

Q100 Why not use the pretrained ASL models? ★★☆

SAY ASL is a different language from ISL, so the signs do not match. We tried PopSign-250 and ASL Citizen on the Pi: they ran, but were not reliable on live input and cannot recognise ISL phrases. So we trained our own.

Q101 Why not a simple classifier like SVM, KNN or random forest? ★★☆

SAY They work for fixed-length, static data. Signs are sequences, so we compared against DTW (a nearest-neighbour method for sequences, 98.5 %). The neural network was slightly better (99.2 %) and gives probabilities.

Q102 **Why MQTT and not HTTP polling or REST between the Pi and the ESP32?** ★★☆

SAY With HTTP the ESP32 would have to keep asking the Pi (polling), which wastes power and bandwidth and adds delay. MQTT pushes the message instantly, has a tiny header, supports QoS, retained messages and last will, and one event can reach many devices.

Q103 **Why not CoAP?** ★★★

SAY CoAP is a good lightweight protocol too, but it is request/response over UDP and has less tooling and no broker-style fan-out. MQTT's broker, retained messages and last will matched our needs and are widely supported on the ESP32.

Q104 **Why not WebSockets?** ★★★

SAY A WebSocket gives a persistent two-way connection, but we would have to build topics, delivery guarantees, retained state and last will ourselves. MQTT already provides them. (WebSockets are good for browser to server live data.)

Q105 **Why not Bluetooth (BLE) or ESP-NOW between the Pi and the ESP32?** ★★★

SAY BLE has short range and a more complex setup on the Pi. ESP-NOW is fast and needs no router, but only between ESP chips, so the Pi could not take part directly. Wi-Fi and MQTT work with the phone, Pi and ESP32 alike on the existing network.

Q106 **Why not LoRa or Zigbee?** ★★★

SAY LoRa is for long range and tiny data rates, and Zigbee needs a hub. We have a short-range, local, fairly chatty system, and every device already has Wi-Fi.

Q107 **Why a local Mosquitto broker and not a cloud broker (HiveMQ, AWS IoT, Adafruit IO)?** ★★☆

SAY A cloud broker needs internet, adds delay and may cost money, and sends our data out. A local broker is free, works offline and fits the edge idea. A cloud broker would allow remote monitoring, which could be future work.

Q108 **Why not MQTT over TLS?** ★★☆

SAY It adds certificate handling on the ESP32 and was beyond our time, and the system is for a private network. We list it as the first security improvement (port 8883).

Q109 **Why ESP32 and not Arduino Uno or ESP8266?** ★★☆

SAY The Uno has no Wi-Fi. The ESP8266 has Wi-Fi but is single-core and has fewer pins. The ESP32 has dual cores, Wi-Fi, Bluetooth and enough GPIO for the OLED, three LEDs, a buzzer and a button, and it is cheap.

Q110 **Why an OLED and not a 16x2 LCD or a TFT?** ★★☆

SAY The OLED is small, has high contrast, needs only two I2C wires and can show graphics like our confidence bar. A 16x2 LCD only shows text (and needs a backpack for I2C), a TFT needs more pins and cost. We also had a 0.96 inch OLED available.

Q111 **Why Wi-Fi and not a wired Ethernet connection?** ★★☆

SAY The phone and ESP32 are wireless and we want a portable system. The Pi can be on Ethernet and the rest on Wi-Fi through the same router.

Q112 **Why did you move the LEDs from the Pi to the ESP32?** ★★☆

SAY To have a separate, wireless output node that can sit near the user, to keep the Pi free of extra wiring, and to show genuine device-to-device IoT with only MQTT between them. The Pi GPIO version also worked and is still in the code.

Q113 **Why self-signed HTTPS and not Let's Encrypt?** ★★☆

SAY Let's Encrypt needs a public domain name and internet access, which a local offline Pi does not have. A self-signed certificate gives encryption with a one-time browser warning.

Q114 **Why ONNX Runtime instead of PyTorch or TensorFlow Lite on the Pi?** ★★☆

SAY ONNX Runtime is fast on the Pi CPU and avoids installing the heavy PyTorch library on the server. TFLite would also be fine, but we trained in PyTorch, so ONNX was the direct export.

Q115 **Why do the MediaPipe step on the phone instead of the Pi?** ★★☆

SAY The phone is faster (about 29 fps with GPU, the Pi only 11 to 16), it keeps the video private and it reduces the data sent to a few tens of KB per sign.

Q116 **Why not stream the video to the Pi with WebRTC?** ★★☆

SAY It uses much more bandwidth and Pi CPU, and loses the privacy benefit. Landmarks are about a thousand times smaller than video.

Q117 **Why speech on the phone and not on the Pi (espeak, pyttsx3)?** ★★☆

SAY The Pi 5 has no audio jack, and phone voices sound better. The phone is next to the user anyway.

Q118 **Why not recognise continuous sentences?** ★★★

SAY It needs a different model type (CTC or sequence-to-sequence) and far more annotated data than we had. Isolated phrases were the achievable goal, and the pipeline can be extended.

Q119 **Why Arduino code and not MicroPython or ESP-IDF?** ★★☆

SAY The Arduino framework has ready libraries for the OLED and MQTT and is quick to write. MicroPython is easier but slower and has fewer libraries. ESP-IDF is more powerful but more complex.

Q120 **Why not a database for the data?** ★★☆

SAY The data are a few hundred small files (`.npy` landmark arrays) plus a metadata CSV. A database would add complexity without benefit at this size.

Q121 **Why not Docker or a systemd service for auto start?** ★★☆

SAY We kept it simple with two commands (`nishu-start`, `nishu-stop`). Auto-start at boot, which needs administrator rights, is in our future work.

Q122 **Why not TinyML (run the model on the ESP32 itself)?** ★★★

SAY The model itself is small, but the ESP32 cannot run MediaPipe on camera frames (limited RAM and no good camera pipeline). The ESP32 could in principle run a tiny classifier on already extracted landmarks, but then we would still need a device for the tracking. The Pi and phone split is more practical.

12 Demo script and when things go wrong

IN SIMPLE WORDS The demo is the most important five minutes. Practise it three times with the real hardware. Start the servers before the professor arrives and keep one person on the phone and one on the ESP32.

BEFORE THE VIVA (DO THIS 10 MINUTES EARLY)

1. Power the Pi and make sure it is on the same Wi-Fi as the phone and the ESP32 (2.4 GHz).
2. In the Pi terminal run `nishu-start`. It starts the MQTT broker and the web server and prints the phone address, for example `https://192.168.29.66:8443/`.
3. Power the ESP32. It flashes each LED and beeps, then the OLED should show "WiFi MQTT" and "Waiting for sign".
4. On the phone open the address, accept the certificate warning once, allow the camera, and choose Hands-free. Check the threshold is about 55 %.
5. Stand in good light, head, shoulders and both hands in the picture, hands low to start.

WHAT TO SAY DURING THE DEMO "I raise my hand and sign Thank you. Watch the phone: it found my hands, sent only the landmark numbers to the Pi, got the answer in a few milliseconds and spoke it. Now look at the ESP32: the green LED lit, it beeped, and the OLED shows the phrase and the confidence. Now I press the emergency button: red LED, alarm, and the OLED says HELP. This message travelled through the MQTT broker." Then show the Teach tab (add a phrase) if time permits.

If this happens	It means	Do this
OLED says "Connecting Wi-Fi" or "Wi-Fi not found"	ESP32 cannot join the network	Check the Wi-Fi name and password in the sketch, make sure it is 2.4 GHz, move closer to the router.
OLED says "Connecting to Pi"	Wi-Fi is OK but the broker is unreachable	Run `nishu-start` on the Pi; check the Pi IP matches MQTT_HOST in the sketch (re-flash if it changed).
Red LED blinks slowly	Pi server offline or connection lost	Same as above; check the Pi is powered and on the network.
Phone says camera blocked	Page is not HTTPS or permission denied	Use the https:// address, accept the warning, allow the camera in the browser.
Nothing is recognised	Hands not detected, bad light, or not raised high enough	Step back so both hands are visible, improve the light, raise the hand above the lap, lower the threshold slider a little.
Wrong phrase shown	Similar signs (e.g. Good morning vs Good afternoon)	Repeat the sign clearly; explain this is the known weak pair from the confusion table.
Pi IP address changed	The router gave a new address	Run `hostname -I` on the Pi and use the new address on the phone (and in the ESP32 sketch).

Commands to remember: `nishu-start` starts everything, `nishu-stop` stops everything. `python webapp/test_server.py` tests the server with stored clips. `python iot/test_iot.py` tests the MQTT layer. `python train_phrases.py --person nishu,phone` retrains.

13 Know our files (what each one does)

IN SIMPLE WORDS If a professor says "show me the code that does X", you should know which file to open. You do not need to memorise the code, only what each file is for.

File	What it does
`webapp/server.py`	The Pi web server: HTTPS, serves the page, `/api/predict`, Teach, Retrain, publishes to MQTT.
`webapp/static/app.js`	The phone page logic: camera, MediaPipe in the browser, hands-free trigger, speech, Teach mode.
`webapp/static/index.html, style.css`	The page layout and the dark theme.
`phrase_model.py`	Loads the ONNX model (reloads when the file changes) and builds the 198-value input features.
`train_phrases.py`	Trains the 1D-CNN+GRU and the DTW baseline, runs cross-validation, exports ONNX.
`record_clips.py`	The webcam recorder page for collecting clips.
`fewshot.py, live_phrases.py`	DTW matcher and a guided live test.
`webapp/mqtt_bus.py`	The MQTT publisher inside the server: topics, QoS, retained, last will, telemetry.
`iot/broker.sh`	Starts, stops and checks the Mosquitto broker.
`iot/esp32_oled/esp32_oled.ino`	ESP32 firmware: Wi-Fi, MQTT, OLED, LEDs, buzzer, button.
`iot/WIRING.md, iot/test_iot.py`	Wiring guide and the automated MQTT test.
`iot/gpio_bridge.py`	Older version that drove LEDs from the Pi pins (not used now).
`models/phrases.onnx`	The trained model. `data/` holds the recorded clips.
`nishu-start, nishu-stop`	Our two commands to start and stop the broker and web server.

Q123 **Draw the software architecture.** ★★☆

SAY Phone page (HTML/JS + MediaPipe) to HTTPS web server on the Pi (server.py + phrase_model.py + ONNX model). The server publishes through mqtt_bus.py to Mosquitto. The ESP32 firmware subscribes to the broker and drives the OLED, LEDs and buzzer, and publishes alerts.

Q124 **Where does the model run and where is it stored?** ★★☆

SAY On the Pi, loaded by `phrase_model.py` with ONNX Runtime from `models/phrases.onnx`. It reloads automatically when the file changes after a retrain.

14 Quick-fire round and trap questions

IN SIMPLE WORDS Short answers to the questions that always come up. Read them out loud once so they sound natural.

Q125 What is the full form of ISL and ASL? ★☆☆

SAY Indian Sign Language and American Sign Language.

Q126 Who maintains ISL? ★☆☆

SAY ISLRTC, the Indian Sign Language Research and Training Centre under the Government of India.

Q127 What is a landmark? ★☆☆

SAY A tracked point on the body, for example a wrist or fingertip, with x and y coordinates.

Q128 What is the output of your model? ★☆☆

SAY 12 probabilities, one per phrase, that sum to 1.

Q129 What is softmax? ★☆☆

SAY A function that turns raw scores into probabilities that add up to 1.

Q130 What is an epoch? ★★☆

SAY One full pass of the training data through the network. We train 80 epochs.

Q131 What is a confusion matrix? ★★☆

SAY A table of true phrase versus predicted phrase. It shows which phrases get mixed up (Good morning with Good afternoon and Sorry).

Q132 What does TCP guarantee? ★☆☆

SAY Reliable, ordered delivery. MQTT and HTTP both run on TCP.

Q133 What is an IP address and a port? ★☆☆

SAY The address of a device on the network, and the numbered door of a program on it.

Q134 What is the difference between MQTT and HTTP? ★★☆

SAY MQTT is publish/subscribe through a broker, light and push-based; HTTP is request/response and the client has to ask.

Q135 What is a topic wildcard? ★★☆

SAY `#` matches everything below a topic, so `signtalk/#` listens to all our topics. `+` matches one level.

Q136 What is an interrupt? Did you use one? ★★☆

SAY A hardware signal that pauses the program to handle an event. We used polling with a 50 ms debounce for the button instead, which is simple and enough.

Q137 What is PWM? ★★☆

SAY Pulse-width modulation: switching a pin on and off very fast. It makes a tone for a passive buzzer or dims an LED. Our default buzzer is active, so we just switch it on and off.

Q138 Why 3.3 V and not 5 V on the ESP32 pins? ★☆☆

SAY The ESP32 logic is 3.3 V. 5 V on a pin can damage it, so the OLED is powered from 3V3.

Q139 What is a watchdog? Do you have one? ★★☆

SAY A timer that restarts the chip if the program hangs. The ESP32 core has a built-in watchdog, and our loop never blocks so it does not trigger. We did not add extra watchdog code.

Q140 How do you power the system? ★★☆

SAY The Pi from its USB-C supply and the ESP32 from USB (5 V into its on-board 3.3 V regulator). A battery or power bank could power the ESP32.

Q141 What if two phones use the system at once? ★★★

SAY The server is multi-threaded and each request takes a few ms, so both get answers; the ESP32 would show whichever phrase arrives last. We tested one phone at a time.

Q142 How do you know the model is not just recognising the signer's shirt or background? ★★★

SAY Because it sees only landmark numbers, not pixels, so clothes and background cannot be used. It could still learn quirks of one person's movement, which is why more signers are needed.

Q143 What would you do first with an extra month? ★★★

SAY Record more signers and test with people who know ISL; that tells us the real accuracy. Second, secure MQTT with TLS.

Q144 What is the most difficult problem you faced? ★★☆

SAY Getting reliable live recognition: pretrained ASL models failed on live camera, so we switched to our own ISL model with landmark features, hands-free windowing and a lower threshold. Also the ESP32 Wi-Fi reconnect bug.

Q145 What did you learn? ★★☆

SAY How a complete IoT chain fits together (sensing, edge AI, MQTT, actuation), how to train and evaluate a small sequence model honestly, and how much real-world details like HTTPS, Wi-Fi bands and debouncing matter.

15 Cute glossary (one line each)

IN SIMPLE WORDS If you hear a word you do not know in the viva, it is probably here.

IoT Internet of Things: connected devices that sense and act.

Edge Computing close to the data source.

Broker The MQTT server that routes messages.

Topic Name of an MQTT message channel.

QoS Quality of Service, MQTT delivery level 0/1/2.

Retained Broker keeps the last message for new subscribers.

LWT Last Will and Testament, broker publishes it if a client dies.

Keep-alive Heartbeat interval of an MQTT client.

TLS Encryption layer used by HTTPS.

Self-signed Certificate signed by yourself, not by an authority.

SSID Wi-Fi network name.

DHCP Automatic IP address assignment.

REST Style of web API using normal URLs.

JSON Text format for data.

GPIO General purpose input/output pin.

I2C Two-wire bus for small devices.

Pull-up Resistor holding an input HIGH.

Debounce Ignoring button bounce.

OLED Display where every pixel makes its own light.

SSD1306 OLED driver chip.

MediaPipe Google's landmark tracking framework.

Landmark A tracked body or hand point.

CNN Convolutional neural network, finds patterns.

GRU Recurrent unit that remembers a sequence.

DTW Dynamic Time Warping, elastic sequence matching.

Softmax Scores to probabilities.

Epoch One pass through the training data.

Overfitting Memorising training data.

Cross-validation Rotating train/test splits for honest accuracy.

Augmentation Making extra training samples by small changes.

ONNX Standard model file format.

Inference Using a trained model to predict.

Threshold Minimum confidence to accept an answer.

LAST CHECKLIST ☐ Everyone can say the 30-second pitch. ☐ Everyone can draw Figure 1 (architecture) and Figure 3 (wiring). ☐ We know our numbers (12 phrases, 603 clips, 99.2 %, 2 to 3 ms). ☐ We know our limits (one signer, no deaf test, no TLS). ☐ We know the alternatives round (Topic 11). ☐ Demo practised three times. ☐ Charged phone, power cables, Wi-Fi password written down. ☐ Smile, speak slowly and say "good question" before answering.